

Code-to-concept mapping for the morphology-conditioned AMP-PPO learning loop in hhi

Scope of the code analysis

The repository at GitHub ¹ (the `hhi` repo owned by hansen1416 ²) is structured around a fork of an ASE/AMP-style imitation-learning stack, with separate modules for (i) motion libraries and datasets, (ii) Isaac-Gym-style humanoid environments, and (iii) PPO + AMP training agents. ³

To match your step-by-step description (actor-critic with PPO plus an AMP discriminator; morphology conditioning via “gender + betas”), the most direct execution path in this repository is:

- **Environment side:** `ase/env/tasks/humanoid.py` (multi-template SMPL humanoids + per-env betas) and `ase/env/tasks/humanoid_phc.py` (PHC tracking + AMP features + shape-conditioned demo sampling). ⁴
- **Learning side:** `ase/learning/phc_agent.py` (PPO rollout, discriminator batches, AMP reward shaping, joint loss, replay buffer). ⁵
- **Motion data side:** `ase/utils/motion_lib_humos.py` (loads motions with gender/beta metadata; builds mapping from shape → valid motion IDs; provides motion-state sampling). ⁶

Everything below maps your conceptual “Step 1 ... Step 14” to concrete blocks in these files.

Morphology-conditioned motion data pipeline

Your Step 1 (“each motion is associated with a body shape condition ... same motion can appear with different morphology settings”) is represented in code by (a) how shapes are discovered from motion files, and (b) how the motion library stores and samples motions conditionally by shape keys. ⁷

How morphology keys are derived

`ase/env/tasks/humanoid.py` builds `loaded_gender_beta_key` by scanning motion file names and extracting `(gender, beta_key)` from each filename stem, then looking up the corresponding SMPL betas via an `all_betas` file under the asset root. ⁸

Conceptually, this establishes the allowed morphology templates the simulator will instantiate:

- shape key: `(gender, beta_key)`
- shape vector: `encode_gender(gender)` concatenated with `betas` (giving an **11-D** vector in this codebase: 1 “gender code” + 10 SMPL betas). ⁹

How motions carry shape metadata and become shape-conditional samples

In `ase/utils/motion_lib_humos.py`, motion loading attaches `gender_beta` to each loaded motion (the same 11-D conditioning vector), and `load_motions()` builds a dictionary mapping each `(gender, beta_key)` to the set of motion IDs that match that shape key. ¹⁰

Critically, `sample_motions(n, gender_beta_keys=...)` implements **shape-conditional motion ID sampling** by drawing from `beta_key_motion_id_mapping[gbk]` for each environment's `gbk`; when `gender_beta_keys` is `None`, it samples uniformly over all motions (used to diversify “real” samples). ¹¹

Interpretation note (faithful to code, but dataset-dependent): the repository *enables* “the same semantic motion existing under multiple morphologies” by allowing multiple motion files with different `(gender, beta_key)` keys. Whether the same motion appears across shapes depends on how your `.pkl` motion set is constructed and named. ¹²

Environment resets and observation construction

Your Steps 2 and 3 align with how the environment (i) instantiates per-env morphologies and (ii) produces two different observation streams: **policy obs** vs **AMP discriminator obs**, plus explicit **shape conditioning**.

Per-env morphology state in the simulator

In `ase/env/tasks/humanoid.py`, environment creation cycles through the available SMPL humanoid assets (one per `(gender, beta_key)` template), and stores for each environment index `i`:

- `env_id_beta_keys_map[i] = (gender, beta_key)`
- `_betas_env[i] = template_beta_vector_for_that_asset` (the 11-D gender+betas vector) ¹³

This is the simulator-side representation of your “morphology condition.” ¹⁴

Reset-to-reference motion state

In `ase/env/tasks/humanoid_phc.py`, `_reset_envs(env_ids)` implements “reset to a state sampled from a reference motion,” including morphology-consistent motion selection:

- for each env id, build `gender_beta_key` by indexing `env_id_beta_keys_map`
- sample a motion consistent with that shape via `motion_lib.sample_motions(num_envs, gender_beta_key)`
- sample a time within that motion (with AMP-history truncation logic)
- query the motion state and write it into simulator tensors (root pose, joint pose, velocities), then push to Isaac Gym tensors. ¹⁵

This is precisely your Step 2 “physically meaningful motion state rather than random pose.” ¹⁶

Policy observation includes task features and betas

In `ase/env/tasks/humanoid_phc.py`, `post_physics_step()` computes tracking features (`task_obs`) and then calls `_compute_observations(task_obs=task_obs)` before computing reward and termination. ¹⁷

Inside `_compute_observations`, the final policy observation is assembled as:

- humanoid state features (from `_compute_humanoid_obs`)
- concatenated with task features (`task_obs`) when enabled
- concatenated with per-env betas (`_betas_env`) and stored in `self.obs_buf`. ¹⁸

This matches your Step 3.1/3.2 claim that actor and critic receive (state, task, morphology). ¹⁹

Implementation detail worth noting: the function computes `betas_norm` but concatenates the unnormalized `betas` into the observation buffer. If you intended normalization, the current code path does not apply it. ²⁰

AMP discriminator features and explicit shape channels

`HumanoidPHC.post_physics_step()` publishes AMP discriminator inputs through the IsaacGym “extras/info” path:

- `extras["amp_obs"] = amp_obs_flat`
- `extras["amp_shape"] = self._betas_env` ²¹

Additionally, `HumanoidPHC.fetch_amp_obs_demo()` generates “real” AMP samples by sampling motions consistent with each env’s morphology key, building AMP features from reference motion states, and returning both `(amp_obs_demo_flat, amp_shape_demo)`. ²²

This directly instantiates your Step 3.3 “discriminator learns $D(x, m)$ ” rather than $D(x)$. ²³

Rollout collection and environment stepping

Your Steps 4, 5, and 7 are implemented primarily in `ase/learning/phc_agent.py` in `play_steps()` / `play_steps_rnn()` and `get_action_values()`.

Actor action selection

`PHCAgent.get_action_values(obs_dict, rand_action_probs)` performs a model forward pass to produce actions and the standard PPO auxiliary outputs (log-probs, values, means `mu`, stds `sigma`). It also implements an **eps-greedy “deterministic-vs-stochastic” mixture**: for environments where `rand_action_mask == 0`, it replaces sampled actions with the mean action `mu`. ²⁴

This corresponds to your Step 4 “Actor produces an action.” ²⁵

Environment step and data capture

`PHCAgent.play_steps()` is the standard PPO rollout loop:

- partial reset of done envs via `env_reset(done_indices)`
- store current obs in the experience buffer
- compute actions via `get_action_values(...)`
- call `env_step(actions)` to step physics and receive `(obs, rewards, dones, infos)`
- store rewards, dones, next_obs
- store `infos['amp_obs']` and `infos['amp_shape']` for AMP

- compute `next_values` via critic and mask out bootstrapping through termination transitions using `next_vals *= (1 - terminated)`. ²⁵

That maps to your Step 5 (“Step the simulator”) and Step 7 (“Collect a rollout”). ²⁴

Discriminator training, AMP reward shaping, PPO targets, and replay buffering

Your Steps 6 and 8–13 are all explicit in `ase/learning/phc_agent.py`.

AMP reward computation and reward mixing

After a rollout is collected, `PHCAgent` computes AMP rewards and combines them with environment rewards:

- `_calc_amp_rewards(amp_obs, amp_shape)` returns a dictionary currently containing `disc_rewards`. ²⁶
- `_calc_disc_rewards` converts discriminator “realness” into reward using `prob = sigmoid(logits)` and a shaped reward `-log(max(1 - prob, eps))`, then scales it by `_disc_reward_scale`. ²⁶
- `_combine_rewards(task_rewards, amp_rewards)` computes a **weighted sum**:
- `combined = task_reward_w * task_rewards + disc_reward_w * disc_rewards`. ²⁷

This implements your Step 6 (“ $r_t = r_{\text{task}} + r_{\text{AMP}}$ ”), except the code uses a tunable **linear mixture** rather than a fixed unit-weight sum. ²⁷

Discriminator training objective and conditioning

During training minibatches, the model forward produces discriminator logits for:

- current rollout AMP batch (fake)
- replay AMP batch (fake, older)
- demo AMP batch (real, requires_grad enabled for gradient penalty). ²⁸

The discriminator loss `_disc_loss(disc_agent_logit, disc_demo_logit, obs_demo)` is explicitly BCE-based:

- `BCEWithLogits(fake_logits, 0) + BCEWithLogits(real_logits, 1)` (averaged)
- plus logit-weight L2 regularization (`_disc_logit_reg`)
- plus gradient penalty on demo samples (`_disc_grad_penalty`)
- plus optional discriminator weight decay (`_disc_weight_decay`). ²⁹

Conditioning on morphology is implemented by threading `amp_shape`, `amp_shape_demo`, `amp_shape_replay` into discriminator evaluation (`eval_disc(proc_amp_obs, amp_shape)`), so the learned classifier is shape-conditioned. ³⁰

Critic target and advantages

After rewards are combined, `PHCAgent` computes advantages and returns via:

- `mb_adv = discount_values(dones, values, rewards, next_values)`
- `mb_returns = mb_adv + mb_values`. ³¹

That corresponds to your Step 10: the critic target is a bootstrapped discounted return (often GAE under the hood of `discount_values`, depending on base rl-games configuration), and the training loss regresses values to `return_batch`. ²⁴

Advantages are computed as `returns - values` and optionally normalized **with masking** so only frames with `rand_action_mask == 1` contribute to normalization (aligning with later masked policy loss). ³²

PPO actor objective and joint update

Inside the training step (the “calc_gradients” path), `PHCAgent` computes:

- PPO actor loss via `_actor_loss(old_logp, new_logp, advantage, clip)`
- critic loss via `_critic_loss(...)`
- entropy bonus and action bound penalty
- discriminator loss via `_disc_loss(...)`
- and then constructs the **joint objective**:

$$\mathcal{L} = \mathcal{L}_{\text{actor}} + c_v \mathcal{L}_{\text{critic}} - c_e \mathcal{H} + c_b \mathcal{L}_{\text{bounds}} + c_d \mathcal{L}_{\text{disc}}$$

(with coefficients taken from config). ³³

So your Step 11 and Step 12 (“actor improves advantage under PPO clipping; actor-critic-discriminator updated jointly”) map directly to this loss construction. ³³

Replay buffer for discriminator stability

Replay storage for AMP observations is explicit: the code filters/downsamples AMP samples based on buffer occupancy and `amp_replay_keep_prob`, then stores a dict containing both `amp_obs` and `amp_shape` into `_amp_replay_buffer`. ³⁴

This is your Step 13 (“store policy motion in replay buffer”) and it is shape-aware (stores shape alongside AMP features). ³⁵

Step-by-step mapping from your conceptual loop to concrete code blocks

Step 1: Prepare the training motion data

Code: `ase/env/tasks/humanoid.py` → `_get_loaded_gender_beta_key(...)`; `ase/utils/`

`motion_lib_humos.py` → `load_motion_with_skeleton(...)`, `load_motions()`,
`sample_motions(...)`

- Motion files define `(gender, beta_key)` pairs, used to pick which SMPL templates exist in the run. ⁸
- Each loaded motion gets an 11-D `gender_beta` vector attached; the motion library builds `(gender, beta_key) → motion_ids` for shape-consistent sampling. ³⁶

Step 2: Initialize the humanoid and reset the environment

Code: `ase/env/tasks/humanoid.py` → env creation assigns `_betas_env` and `env_id_beta_keys_map`; `ase/env/tasks/humanoid_phc.py` → `_reset_envs(...)`, `_reset_actors(...)`, `_reset_env_tensors(...)`, `_init_amp_obs(...)`, `_init_amp_obs_ref(...)`

- Per-env morphology is fixed by selecting an SMPL asset and copying its 11-D `(gender+betas)` into `_betas_env[i]` and its `(gender, beta_key)` into `env_id_beta_keys_map[i]`. ¹³
- Reset samples a reference motion consistent with each env's morphology key, samples a time, queries motion state, writes root pose/orientation/velocities and DOF pose/velocities into simulation tensors, then initializes AMP history from reference frames. ³⁷

Step 3: Build the observations for the three components

Step 3.1: Actor observation

Code: `ase/env/tasks/humanoid_phc.py` → `_compute_observations(...)`; `ase/learning/phc_agent.py` → `get_action_values(...)`

- Env concatenates humanoid state + task features + `_betas_env` into `obs_buf`. ¹⁸
- Agent preprocesses `obs_dict['obs']` and feeds it into `self.model(...)` to get actions (policy). ²⁵

Step 3.2: Critic observation

Code: `ase/learning/phc_agent.py` → `get_action_values(...)` and critic evaluation inside `play_steps()`

- The forward pass returns `values`; optionally a "central value" path uses privileged `states` (if enabled). ³⁸
- Next-step bootstrap value is computed and masked by termination: `next_vals *= (1 - terminate)`. ²⁵

Step 3.3: Discriminator observation

Code: `ase/env/tasks/humanoid_phc.py` → `post_physics_step()` publishes `extras["amp_obs"]` and `extras["amp_shape"]`; `fetch_amp_obs_demo()` returns `(amp_obs_demo_flat, amp_shape_demo)`; `ase/learning/phc_agent.py` → rollout stores `infos['amp_obs']`, `infos['amp_shape']`, and training assembles `amp_obs`, `amp_obs_demo`, `amp_obs_replay` with matching `amp_shape*`

- Policy rollout AMP features: `infos['amp_obs']`, `infos['amp_shape']`. ³⁹
- Real demos are sampled shape-consistently from the motion library and returned with `amp_shape_demo`. ⁴⁰
- Replay samples (older fakes) are drawn from `_amp_replay_buffer` and carry `amp_shape_replay`. ⁴¹

Step 4: Actor produces an action

Code: `ase/learning/phc_agent.py` → `get_action_values(...)`

- Model forward returns action tensors; eps-greedy mixing may replace stochastic actions with `mu` for some envs. ²⁵

Step 5: Step the simulator

Code: `ase/learning/phc_agent.py` → `play_steps()` calls `env_step(actions)`; `ase/env/tasks/humanoid_phc.py` → `post_physics_step()` computes obs/reward/reset and populates `extras`

- Agent receives `(obs, rewards, dones, infos)` and stores the transition plus AMP extras. ⁴²

Step 6: Compute the current reward

Code: `ase/env/tasks/humanoid_phc.py` → `_compute_reward(...)` called in `post_physics_step()`; `ase/learning/phc_agent.py` → `_calc_amp_rewards(...)`, `_combine_rewards(...)`

- Task reward is computed inside the environment post-step. ¹⁷
- Discriminator reward is computed from discriminator logits and mixed with task reward via `(task_reward_w, disc_reward_w)`. ⁴³

Step 7: Collect a rollout

Code: `ase/learning/phc_agent.py` → `play_steps()` / `play_steps_rnn()` storing into `experience_buffer`

- Stores obs, actions, log-probs, values, rewards, dones, next_values, plus AMP tensors `amp_obs`, `amp_shape`, and `rand_action_mask`. ⁴⁴

Step 8: Train the discriminator

Code: `ase/learning/phc_agent.py` → discriminator minibatch preparation + `_disc_loss(...)`

- Uses BCE classification losses for fake vs real; adds logit regularization, gradient penalty, and optional weight decay. ⁴⁵
- Conditioning is via passing `amp_shape*` into discriminator evaluation. ⁴⁶

Step 9: Convert discriminator output into AMP reward

Code: `ase/learning/phc_agent.py` → `_calc_disc_rewards(...)` called by `_calc_amp_rewards(...)`

- Converts logit → prob via sigmoid and then applies a `-log(1 - prob)`-style shaping with scale `_disc_reward_scale`. ⁴⁷

Step 10: Compute the critic target

Code: `ase/learning/phc_agent.py` → `discount_values(...)` and `mb_returns = mb_adv + mb_values`; critic loss call in training step via `_critic_loss(...)`

- Returns are computed from rewards and bootstrapped `next_values`, with failure bootstrapping suppressed by `next_vals *= (1 - terminate)`. ²⁴

Step 11: Compute the actor target

Code: `ase/learning/phc_agent.py` → advantages computed as `returns - values` (with optional masked normalization), then PPO clipped surrogate via `_actor_loss(...)`

- Actor loss uses advantage and PPO clipping; loss contributions are masked by `rand_action_mask`. 32

Step 12: Update actor, critic, and discriminator jointly

Code: `ase/learning/phc_agent.py` → joint loss construction in training step (`loss = ... + disc_coef * disc_loss`)

- Total loss combines actor loss, critic regression, entropy, bounds loss, and discriminator loss. 33

Step 13: Store policy motion in replay buffer

Code: `ase/learning/phc_agent.py` → replay-store logic calling `_amp_replay_buffer.store({'amp_obs': ..., 'amp_shape': ...})` with keep-prob downsampling

- Replay buffer stores both AMP features and their associated morphology conditioning vector. 35

Step 14: Repeat

Code: `ase/learning/phc_agent.py` → the `train_epoch()` structure described in the class-level algorithm docstring; run-loop orchestration is performed by the surrounding runner/trainer stack

- The code organizes each epoch as rollout → demo/replay batch attachment → PPO/discriminator minibatch updates → replay append. 48

1 5 24 25 26 27 28 29 30 31 32 33 34 35 38 42 43 44 45 46 47 48 https://github.com/hansen1416/hhi/raw/refs/heads/main/ase/learning/phc_agent.py
https://github.com/hansen1416/hhi/raw/refs/heads/main/ase/learning/phc_agent.py

2 15 16 17 18 19 20 21 22 23 37 39 40 https://github.com/hansen1416/hhi/blob/main/ase/env/tasks/humanoid_phc.py
https://github.com/hansen1416/hhi/blob/main/ase/env/tasks/humanoid_phc.py

3 <https://github.com/hansen1416/hhi>
<https://github.com/hansen1416/hhi>

4 7 8 9 12 13 14 <https://github.com/hansen1416/hhi/blob/main/ase/env/tasks/humanoid.py>
<https://github.com/hansen1416/hhi/blob/main/ase/env/tasks/humanoid.py>

6 10 11 36 https://github.com/hansen1416/hhi/blob/main/ase/utils/motion_lib_humos.py
https://github.com/hansen1416/hhi/blob/main/ase/utils/motion_lib_humos.py

41 https://github.com/hansen1416/hhi/blob/main/ase/learning/phc_agent.py
https://github.com/hansen1416/hhi/blob/main/ase/learning/phc_agent.py